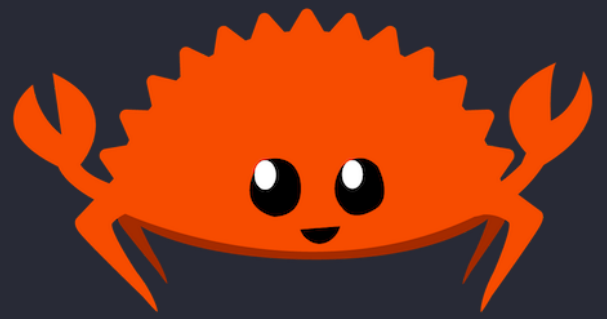
Data Ownership

CS-3180

Rust

- Unique approach to memory management
- Generally, you won't need to worry about allocation
- Allowing data to flow through your program
- Keep track of who it belongs to



Ownership

- *Ownership* is a set of rules that govern how to manage memory
- Prevents need for a garbage collector
- Or `malloc` + `free`
- Ownership is determined entirely at compile time

Ownership Rules:

- 1. Every *value* has an owner
- 2. There can only be *one* owner at a time
- 3. When the owner goes out of *scope* the value will be dropped
- Data is only valid while it is still within scope

Drop

- Once any variable (including our `String`) goes out of scope
- Rust calls the `drop` function on it
- Its *lifetime* has ended
- This also applies when data is re-assigned
- Invalids the data
- Go a little deeper with a more complex type

Heap Data

- Everything we talked about before had a known, fixed size
- Meaning it can be stack allocated
- String *literals* (`&str`) have a fixed size (known at compile time)
- Strings generally (`String`) do not, so they need to be heap allocated

Strings

- `::` is a namespace lookup (instead of a method)
- `String`s can be mutated, literals cannot
- Difference comes down to memory

```
1 fn main() {
2     let mut s: String = String::from("Hello");
3     let literal: &str = ", world!";
4 
5     s.push_str(literal);
6     println!("{s}");
7 }
```

Slices

- `&str` are often called *string slices*
- Immutable view into another string (static literal, stack, or heap)

```
1 fn main() {
2     let my_string = String::from("Rust Memory");
3     let slice: &str = &my_string[0..4];
4     println!("{slice}");
5
6 }
```

String Owners

- When we make a new `String` variable
- Create a *binding* between the label and the data
- That label takes *ownership* of the data
- If we were to attempt to copy this data, something weird happens

```
1 fn main() {
2     let bind_one = String::from("Hello world");
3
4     let bind_two = bind_one;
5     println!("{bind_one}");
6 }
```

Move Semantics

- We got a compile-time error
- "Borrow of moved value: *bind_one*"
- Think back to our ownership rules
- Data can only have one owner at a time
- The previous variable no longer owns the data
- Ownership is transfer, the data is *moved*

How do we fix this?

- Simplest way: Just tell Rust to `clone` the data
- Most types chose to implement `clone()`
- Copying heap data like this is expensive
- Try to avoid it if you can

```
1 fn main() {
2     let bind_one = String::from("Hello world");
3
4     let bind_two = bind_one.clone();
5     println!("{bind_one}");
6 }
```

Borrowing

- Better way: Change the way you take ownership
- When you pass data to a function
- You are assigning the parameters (local variables scoped to the function)
- The data from the caller
- This has some ownership implications
- Let's see that:

Parameter Ownership

- We can't print `s` later
- Because `calculate_length` took ownership of our data

```
1  fn calculate_length(s: String) -> usize {
2      s.len()
3  }
4
5  fn main() {
6      let s = String::from("Hello world");
7
8      calculate_length(s);
9
10     println!("{s}");
11 }
```

Parameter Ownership

- We can't print `s` later
- Because `calculate_length` took ownership of our data

```
1  fn calculate_length(s: String) -> usize {
2      s.len()
3  }
4
5  fn main() {
6      let s = String::from("Hello world");
7
8      calculate_length(s);
9
10     println!("{s}");
11 }
```

Parameter Ownership

- We can't print `s` later
- Because `calculate_length` took ownership of our data

```
1  fn calculate_length(s: String) -> usize {
2      s.len()
3  }
4
5  fn main() {
6      let s = String::from("Hello world");
7
8      calculate_length(s);
9
10     println!("{s}");
11 }
```

Parameter Ownership

- We can't print `s` later
- Because `calculate_length` took ownership of our data

```
1  fn calculate_length(s: String) -> usize {
2      s.len()
3  }
4
5  fn main() {
6      let s = String::from("Hello world");
7
8      calculate_length(s);
9
10     println!("{s}");
11 }
```

Borrowing

- How do we restore ownership?
- We could possibly return the data, but that's not practical
- We need a way of making a reference to the data, without actually taking ownership of it
- Enter: Borrowing

Borrowing

- Denote a parameter as accepting a *borrow* of data
- Using `&`
- `s` is now a reference to String
- We also need to pass it as a reference

```
1 fn calculate_length(s: &String) -> usize {
2     s.len()
3 }
4
5 fn main() {
6     let s = String::from("Hello world");
7     let len = calculate_length(&s);
8     println!("{s} of len: {len}");
9 }
```

Immutable References

- Remember parameters are just local variables
- Same mutability rules apply
- Take a look at this snippet:

```
1  // attempts to create a side-effect
2  fn change(some_string: &String){
3      some_string.push_str(", world!");
4  }
5
6  fn main() {
7      let s = String::from("Hello");
8      change(&s);
9      println!("{s}");
10 }
```

Mutable References

- We need to explicitly tell the compiler that this reference is *mutable*
- Change borrow at callie, denoting side-effects

```
1  // attempts to create a side-effect
2  fn change(some_string: &mut String){
3      some_string.push_str(", world!");
4  }
5
6  fn main() {
7      // since this variable changes, it MUST be marked as mut
8      let mut s = String::from("Hello");
9      change(&mut s);
10     println!("{s}");
11 }
```

Mutable References

- We need to explicitly tell the compiler that this reference is *mutable*
- Change borrow at callie, denoting side-effects

```
1  // attempts to create a side-effect
2  fn change(some_string: &mut String){
3      some_string.push_str(", world!");
4  }
5
6  fn main() {
7      // since this variable changes, it MUST be marked as mut
8      let mut s = String::from("Hello");
9      change(&mut s);
10     println!("{s}");
11 }
```

Mutable References

- We need to explicitly tell the compiler that this reference is *mutable*
- Change borrow at callie, denoting side-effects

```
1  // attempts to create a side-effect
2  fn change(some_string: &mut String){
3      some_string.push_str(", world!");
4  }
5
6  fn main() {
7      // since this variable changes, it MUST be marked as mut
8      let mut s = String::from("Hello");
9      change(&mut s);
10     println!("{s}");
11 }
```

Mutable References

- We need to explicitly tell the compiler that this reference is *mutable*
- Change borrow at callie, denoting side-effects

```
1  // attempts to create a side-effect
2  fn change(some_string: &mut String){
3      some_string.push_str(", world!");
4  }
5
6  fn main() {
7      // since this variable changes, it MUST be marked as mut
8      let mut s = String::from("Hello");
9      change(&mut s);
10     println!("{s}");
11 }
```

Inconsistencies

- Why didn't we have this issue before with ints?
- Why does the following code work?
- Doesn't this violate ownership rules?

```
 1 fn add(a: i32, b: i32) -> i32 {
 2     a + b
 3 }
 4
 5 fn main() {
 6     let a = 5;
 7     let b = 6;
 8
 9     let sum = add(a, b);
10     println!("{a} + {b} = {sum}")
11 }
```

Copy vs. Clone

- Small, fixed-size types are often considered "cheap"
- They're not very expensive to copy the (often stack) data
- Implement a trait under the hood called `Copy`
- Things are cheap, will just copy the data without needing to borrow

Hold on?

- Did we say before that we couldn't use scope as a memory dropping mechanic?
- Yes, but *why* couldn't we do it before?

```
1  // mallocs some data (heap)
2  int* create_heap_integer(int value) {
3      int *ptr = (int*)malloc(sizeof(int));
4      *ptr = value;
5
6      return ptr;
7  }
8  // ...
9  int *my_number = create_heap_integer(42);
10 // ...
```

Scope

- If this were in Rust, the ownership would have been transfer
- Back to the pointer, then back to whoever used it in main (returning transfers ownership)

```
1 int* create_heap_integer(int value) {
2     int *ptr = (int*)malloc(sizeof(int));
3     *ptr = value;
4 
5     return ptr;
6 }
7 
8 int *my_number = create_heap_integer(42);
```

Scope

- If this were in Rust, the ownership would have been transfer
- Back to the pointer, then back to whoever used it in main (returning transfers ownership)

```
1 int* create_heap_integer(int value) {
2     int *ptr = (int*)malloc(sizeof(int));
3     *ptr = value;
4 
5     return ptr;
6 }
7 
8 int *my_number = create_heap_integer(42);
```

Scope

- If this were in Rust, the ownership would have been transfer
- Back to the pointer, then back to whoever used it in main (returning transfers ownership)

```
1 int* create_heap_integer(int value) {
2     int *ptr = (int*)malloc(sizeof(int));
3     *ptr = value;
4 
5     return ptr;
6 }
7 
8 int *my_number = create_heap_integer(42);
```

Scope

- If this were in Rust, the ownership would have been transfer
- Back to the pointer, then back to whoever used it in main (returning transfers ownership)

```
1 int* create_heap_integer(int value) {
2     int *ptr = (int*)malloc(sizeof(int));
3     *ptr = value;
4 
5     return ptr;
6 }
7 
8 int *my_number = create_heap_integer(42);
```

Summary

- Rust memory management is governed by a set of rules
- Parameters steal ownership of data
- Pass references with &
- Mutability rules apply
- Small fixed-size stack data copies data automatically

Questions?